Carleton Smart Course Scheduler

SYSC 4907 Capstone Project

Helping students check prerequisites and build conflict-free schedules

January 2026

The Problem

Course registration at Carleton is a pain

Checking Prerequisites

- Open your audit, open the calendar, go back and forth
- “Wait, do I have SYSC 3303 or not?”
- Takes a while if you’re picking 5-6 courses

Info is Everywhere

- Carleton Central for one thing
- Course calendar for another
- Department sites for elective lists

Building a Schedule

- No way to check conflicts automatically
- Try a section, doesn’t work, try another
- Hope you don’t mess up LEC/LAB/TUT

Actually Registering

- Hunt down CRNs one by one
- Copy them into Central
- Pray nothing conflicts

The Old Way: Checking Prerequisites

Cross-referencing your audit with the course calendar

1. Your Course Audit

SUMMARY OF CREDITS AND CGPA
Engineering Program Only

EARNED:	17.00 Credits	
	16.50 CGPA Credits	145.00 Points | 8.78 CGPA
IP	2.00 Credits	
NEEDS:	2.00 Credits	5.00 CGPA

NO COMPLEMENTARY STUDIES, CCDP 2100...

EARNED: 1.00 Credit
NEEDS: 0.50 Credits

What courses have I completed?

2. Course Calendar

SYSC 2004 [0.5 credit]
Object-Oriented Software Development

Prerequisite(s): SYSC 1006 or SYSC 2006 or permission of the department, and second-year status in Engineering.

SYSC 2006 [0.5 credit]
Foundations of Imperative Programming

Prerequisite(s): Second-year status in Engineering.

What do I need to take this course?

Students must manually cross-reference these two sources for every course they want to take

The Old Way: Building a Schedule

Conflicts only appear after you try to register

Winter 2026

Winter Schedule

Time	Mon	Tue	Wed	Thu	Fri
10:00			SYSC 4130 **Full section**		
11:30		SYSC 2004 E **Full section**	SYSC 3303 A LEC	SYSC 2004 E **Full section**	SYSC 3303 A LEC
15:00	SYSC 2004 D LEC, 14753	SYSC 2004 L4 LAB, 14758	SYSC 2004 D LEC, 14753		
16:30			SYSC 4416 L1 LAB		SYSC 2004 L3 LAB, 14757

Red

= Conflict / Full Section

The Problem

- You pick your courses and sections
- Build what looks like a good schedule
- Try to register…
- **Conflict!** Two classes overlap
- Or worse: section is **full**

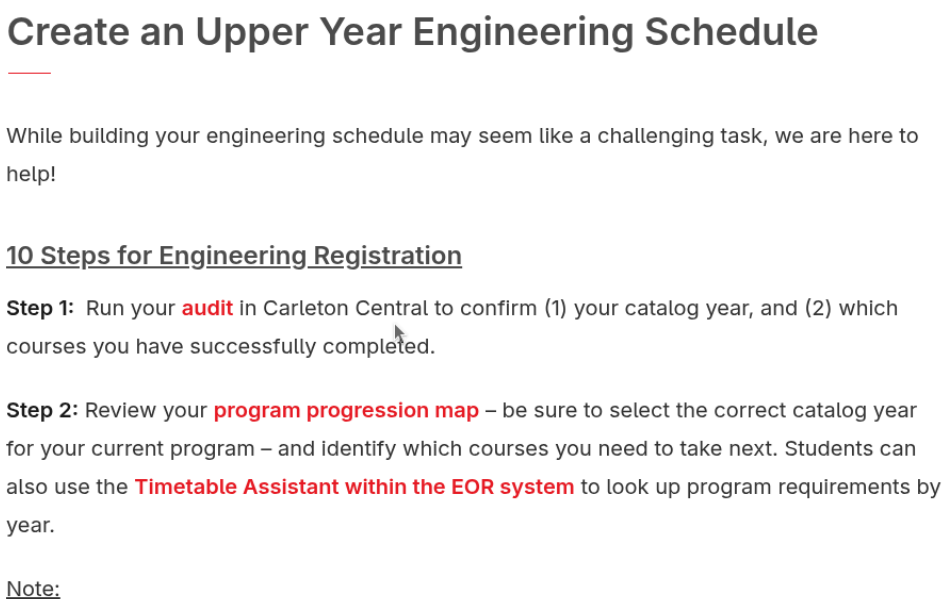
Result: Start over with different sections, hope they don't conflict either

No Way to Know Until You Try

- No automatic conflict detection
- Must manually check every time slot
- LEC + LAB + TUT all need to fit

The Old Way: Info is Everywhere

10 steps just to register for courses?



Create an Upper Year Engineering Schedule

While building your engineering schedule may seem like a challenging task, we are here to help!

10 Steps for Engineering Registration

Step 1: Run your **audit** in Carleton Central to confirm (1) your catalog year, and (2) which courses you have successfully completed.

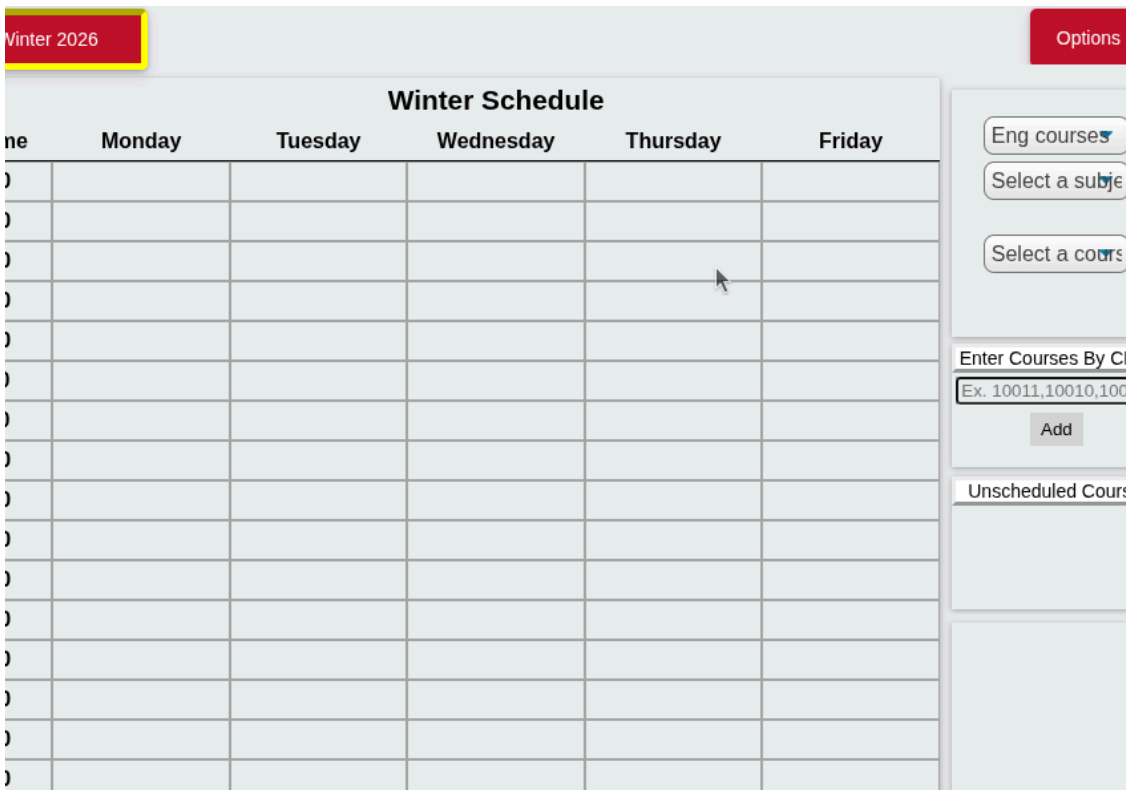
Step 2: Review your **program progression map** – be sure to select the correct catalog year for your current program – and identify which courses you need to take next. Students can also use the **Timetable Assistant within the EOR system** to look up program requirements by year.

Note:

Students must navigate multiple disconnected systems just to build a schedule

The Old Way: No Search

You can't just type a course name



The Problem

- No search bar at all
- Must click through dropdown menus
- "Select a subject" → "Select a course"
- Or enter CRNs manually if you know them

What This Means

- Can't type "SYSC 4001" and find it
- Have to browse through long lists
- Slows down the whole process

Even basic search functionality is missing from Carleton's scheduler

Our Solution: Engineering Overview

A full-stack system that consolidates fragmented data and automates manual processes

System Architecture

Frontend — React + TypeScript
Audit upload, course search, schedule viewer

↓ REST API ↓

Backend — Express + Node.js
Validation engine, schedule generator

↓ Prisma ORM ↓

Database — PostgreSQL
Courses, prerequisites, sections, students

Core Engineering Problems

1. Academic Audit Parsing
Extract structured data from unstructured HTML

2. Prerequisite Representation
Model complex AND/OR logic as evaluable trees

3. Schedule Generation
Constraint satisfaction with backtracking

4. Data Acquisition
Scraping, electives, multi-term support

Next: Implementation walkthrough — what we built (in order) and why

Section 1: Academic Audit Parsing

The foundation — everything else depends on knowing what you've completed

Why This Comes First

Before we can do anything useful, we need to know:

- Which courses has the student completed?
- What grades did they get?
- What program are they in?
- What requirements remain unfulfilled?

Every other feature depends on this data:

- Prerequisite validation needs completed courses
- Elective suggestions need program info
- Schedule generation needs to know what's left

The Challenge

No API exists.
Carleton Central provides audits as HTML pages only.
We must extract structured data from unstructured markup.

What We Extract

- Student ID (9 digits)
- Student name
- Program (e.g., "Software Engineering")
- Completed courses with grades
- In-progress courses (current term)
- Requirement status (fulfilled/unfulfilled)

Audit Parser: Data Structures

TypeScript interfaces that model the extracted data

StudentAuditData Interface

```
interface StudentAuditData {
  studentId: string      // "123456789"
  name: string           // "John Smith"
  program: string        // "Software Engineering"

  completedCourses: Array<{
    courseId: string     // "SYSC3303"
    grade: string        // "A-"
  }>

  currentCourses: Array<{
    courseId: string     // "SYSC4907"
    term: string         // "25WN"
  }>

  requirements: RequirementInfo[]
}
```

RequirementInfo Interface

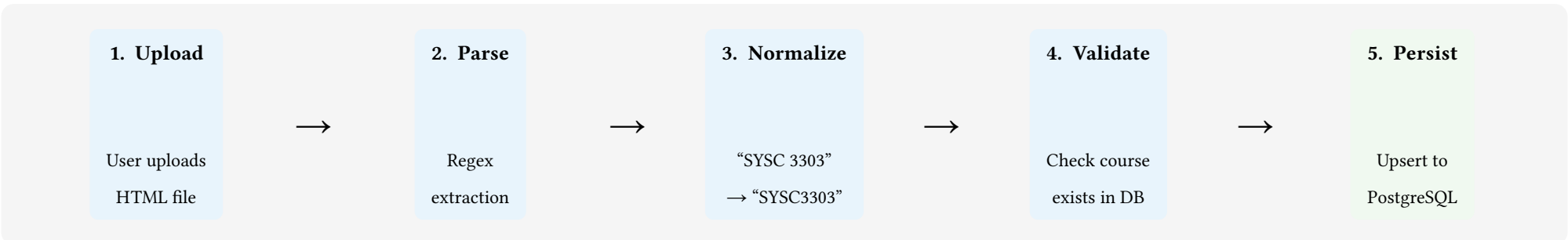
```
interface RequirementInfo {
  title: string
  status: 'OK' | 'NO' | 'IP' | 'NONE'
  creditsRequired: number
  creditsCompleted: number
  creditsInProgress: number
  courseOptions: string[]
}
```

Why These Fields?

- **status**: track completion state
- **creditsRequired vs Completed**: calculate progress
- **courseOptions**: which courses satisfy requirement

Audit Parser: Pipeline Architecture

From raw HTML upload to structured database records



Key Design Decision: Upsert

- Use `prisma.student.upsert()` to handle re-uploads
- If student exists → update their data
- If new student → create record
- Idempotent: safe to upload same audit multiple times

Database Operations

1. Upsert Student record
2. Delete old CourseCompletions (clean slate)
3. Insert new CourseCompletions
4. Link to existing Course records in DB

Audit Parser: Regex Patterns

Targeted patterns for each data field we need to extract

Core Extraction Patterns

Field	Regex Pattern	Example Match
Student ID	`/(\d{9})/`	“123456789”
Name	`/([A-Z][a-z]+), ([A-Z][a-z]+)/`	“Smith, John” ⟶ “John Smith”
Program	`/Bachelor of Engineering - ([^<]+)/`	“Software Engineering”
Course Row	`/<tr[^>]*class="takenCourse"[^>]*>/g`	Matches each course row

Course Extraction (Complex)

```
/<tr[^>]*class="takenCourse"[^>]*>
  [\s\S]*?<td[^>]*class="term"[^>]*>
    ([^<]*)<\/td>
  [\s\S]*?<td[^>]*class="course"[^>]*>
    ([A-Z]{3,4}\s+\d{4})<\/td>
  [\s\S]*?<td[^>]*class="grade"[^>]*>
    ([^<]+)<\/td>/g
```

Why This Pattern?

- `[\s\S]*?` — non-greedy match across newlines
- Captures: term, course code, grade
- Class selectors target stable HTML structure
- Global flag `/g` finds all course rows

Audit Parser: Requirement Extraction

Parsing subrequirements to identify what's still needed

HTML Structure We Parse

```
<div class="subrequirement"
     rqdHours="0.50">
  <span class="status Status_NO">
  <span class="subreqTitle">
    COMPLEMENTARY STUDIES, CCDP 2100
  </span>
  ...
  <span class="hours">0.00</span>
  cr. Complete
</div>
```

We extract: required credits, completed credits, status, and course options from each subrequirement block.

Status Detection Logic

```
// Determine status from CSS class
let status: 'OK'|'NO'|'IP'|'NONE' = 'NO'
if (statusClass.includes('OK'))
  status = 'OK'
else if (statusClass.includes('IP'))
  status = 'IP'
```

Elective Category Detection

- "complementary studies" ⟶ `COMPLEMENTARY_STUDIES`
- "computer science elective" ⟶ `CS_ELECTIVE`
- "basic science" ⟶ `SCIENCE_ELECTIVE`
- These map to lists of valid courses later

Audit Parser: PDF Support

Supporting both HTML exports and PDF text extraction

PDF Line Format

```
21FA   CHEM 1101   0.50   A+
21WN   MATH 1004   0.50   B
25WN   SYSC 4907   0.50   CUR
```

PDF Pattern

```
/^\s*(\d{2}(?:FA|WN|SU))\s+
  ([A-Z]{3,4})\s+(\d{4})\s+
  ([\d.]+)\s+([A-Z+\-]+)/gm
```

Grade Filtering Logic Not all rows are valid completions:

```
// Current course (not completed yet)
if (grade === 'CUR') {
  currentCourses.push({ courseId, term })
}
// Failed or withdrawn - skip
else if (['F','X','UNS','NR'].includes(grade)) {
  continue
}
// Valid completion
else {
  completedCourses.push({ courseId, grade })
}
```

Key: We use `Set` to deduplicate — same course may appear multiple times if retaken.

Audit Parser: Database Persistence

Saving parsed data with Prisma ORM

Student Upsert

```
const student = await prisma.student.upsert({
  where: { studentId: auditData.studentId },
  update: {
    name: auditData.name,
    program: auditData.program
  },
  create: {
    studentId: auditData.studentId,
    name: auditData.name,
    program: auditData.program
  }
})
```

Upsert = Update if exists, Insert if new
Handles re-uploads gracefully

Course Completion Storage

```
// Clean slate for this student
await prisma.courseCompletion.deleteMany({
  where: { studentId: student.id }
})

// Insert all completions
for (const completion of completedCourses) {
  const course = await prisma.course.findUnique({
    where: { id: completion.courseId }
  })
  if (course) {
    await prisma.courseCompletion.create({
      data: {
        studentId: student.id,
        courseId: completion.courseId,
        grade: completion.grade
      }
    })
  }
}
```

Audit Parser: Trade-offs & Limitations

Design decisions and their consequences

Approach Comparison

Approach	Pros	Cons
Regex	Fast, simple, no deps	Brittle to HTML changes
DOM (cheerio)	Robust, semantic	Heavier, slower
Official API	Reliable	Doesn't exist

Why We Chose Regex

- O(n) single-pass parsing
- No external dependencies
- Patterns target stable class names
- Easy to update if structure changes

Known Limitations

1. **HTML structure dependency** — if Carleton redesigns their audit page, patterns break
2. **No real-time sync** — student must re-upload after completing courses
3. **Course existence check** — courses not in our DB are skipped (logged as warnings)

Mitigation Strategy

- Fallback patterns for common variations
- Comprehensive logging for debugging
- Could migrate to DOM parser if needed

Section 2: Prerequisite Representation

Modeling complex AND/OR logic for automated validation

The Core Problem

Prerequisites aren't simple lists. They have **logic**:

> **SYSC 4001:**
> "SYSC 3303 **and** (SYSC 3310 **or** SYSC 3320)"
>
> **SYSC 4805:**
> "Fourth-year standing **or** permission"

We need to:

- Store this logic in a database
- Evaluate it against a student's transcript
- Show clear feedback (pass/fail + what's missing)

Requirements

1. **Persist in relational DB** — can't just store strings
2. **Evaluate programmatically** — no human interpretation
3. **Support multiple node types:**
   - Course requirements (SYSC3303)
   - Year standing (Fourth-year)
   - Special permission
   - Grade requirements (C- minimum)
4. **Handle nested logic** — AND inside OR inside AND
5. **Provide actionable feedback** — list missing courses

Prerequisite Schema: Database Design

Self-referential tree structure in PostgreSQL via Prisma

PrerequisiteNode Model

```
model PrerequisiteNode {
  id               String   @id @default(cuid())
  courseId         String   // Which course this prereq belongs to
  nodeType         String   // AND | OR | COURSE | YEAR_STANDING | PERMISSION

  // For COURSE nodes
  requiredCourseId String?  // FK to Course
  requiredGrade    String?  // e.g., "C-"

  // For YEAR_STANDING nodes
  requiredYear     String?  // FIRST_YEAR | SECOND_YEAR | etc.

  // Tree structure (self-reference)
  parentId         String?
  parent           PrerequisiteNode?  @relation("NodeHierarchy")
  children         PrerequisiteNode[] @relation("NodeHierarchy")

  // Relations
  course           Course   @relation("CoursePrerequisites")
  requiredCourse   Course?  @relation("RequiredByCourses")
}
```

Why Self-Reference?

- Trees need parent-child relationships
- `parentId` points to another node in same table
- Root nodes have `parentId = null`
- Prisma handles the FK automatically

Node Types

Type	Meaning
AND	All children must pass
OR	Any child must pass
COURSE	Specific course required
YEAR_STANDING	Minimum year level
PERMISSION	Manual override needed

Prerequisite Parser: Expression → Tree

Converting human-readable expressions to node trees

Input Format (from course calendar)

```
"SYSC3303 && (SYSC3310 || SYSC3320)"
"FOURTH_YEAR || PERMISSION"
"COMP1006[C-] && MATH1004"
```

Parser Strategy

1. Split on && first (lower precedence)
2. Split on || within groups
3. Handle parentheses for nesting
4. Parse individual terms (courses, years, etc.)

Parsing Logic

```
function parsePrerequisiteExpression(expr: string): ParsedNode {
  // Handle simple AND (no OR present)
  if (expr.includes(' && ') && !expr.includes(' || ')) {
    const parts = expr.split(' && ')
    return {
      type: 'AND',
      children: parts.map(parseSimpleTerm)
    }
  }

  // Handle simple OR (no AND present)
  if (expr.includes(' || ') && !expr.includes(' && ')) {
    const parts = expr.split(' || ')
    return {
      type: 'OR',
      children: parts.map(parseSimpleTerm)
    }
  }

  // Handle parentheses - recursive
  if (expr.includes('(')) {
    return parseComplexExpression(expr)
  }

  return parseSimpleTerm(expr)
}
```

Prerequisite Parser: Term Recognition

Identifying different types of requirements

parseSimpleTerm Function

Example Transformations

Input	Output Node
`"SYSC3303"`	`{type: 'COURSE', value: 'SYSC3303'}`
`"COMP1006[C-]"`	`{type: 'COURSE', value: 'COMP1006', grade: 'C-'}`
`"FOURTH_YEAR"`	`{type: 'YEAR_STANDING', value: 'FOURTH_YEAR'}`
`"PERMISSION"`	`{type: 'PERMISSION'}`

```
function parseSimpleTerm(term: string): ParsedNode {
  term = term.trim().replace(/[()]/g, '')

  // Year standing
  if (['FIRST_YEAR', 'SECOND_YEAR',
       'THIRD_YEAR', 'FOURTH_YEAR'].includes(term)) {
    return { type: 'YEAR_STANDING', value: term }
  }

  // Permission
  if (term === 'PERMISSION') {
    return { type: 'PERMISSION' }
  }

  // Course with grade requirement
  const gradeMatch = term.match(
    /^([A-Z]{3,4}\d{4})\[([A-Z\-+]*)\]$/
  )
  if (gradeMatch) {
    return {
      type: 'COURSE',
      value: gradeMatch[1],  // COMP1006
      grade: gradeMatch[2]   // C-
    }
  }

  // Standard course
  if (term.match(/^[A-Z]{3,4}\d{4}$/)) {
    return { type: 'COURSE', value: term }
  }

  // Unknown - treat as permission
  return { type: 'PERMISSION' }
}
```

Fallback behavior: Unknown terms become PERMISSION nodes to avoid foreign key errors (logged as warnings).

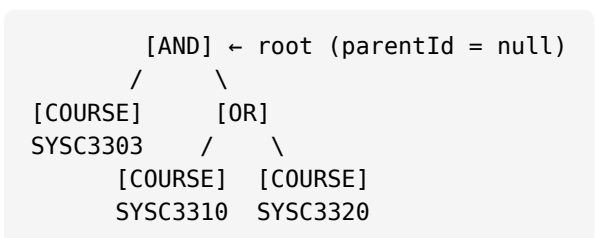
Prerequisite Storage: Building the Tree

Recursively creating nodes in PostgreSQL

Recursive Tree Creation

Visual Example

Expression: `"SYSC3303 && (SYSC3310 || SYSC3320)"`



Each box = one row in `prerequisite_nodes` table
Links via `parentId` foreign key

```
async function createPrerequisiteTree(
  courseId: string,
  parsedNode: ParsedNode,
  parentId?: string
): Promise<string> {

  // Create this node
  const node = await prisma.prerequisiteNode.create({
    data: {
      courseId,
      nodeType: parsedNode.type,
      requiredCourseId: parsedNode.type === 'COURSE'
        ? parsedNode.value : null,
      requiredYear: parsedNode.type === 'YEAR_STANDING'
        ? parsedNode.value : null,
      requiredGrade: parsedNode.grade || null,
      parentId: parentId || null
    }
  })

  // Recursively create children
  if (parsedNode.children) {
    for (const child of parsedNode.children) {
      await createPrerequisiteTree(
        courseId, child, node.id
      )
    }
  }

  return node.id
}
```

Prerequisite Validation: Evaluation Algorithm

Recursive tree traversal with boolean logic

Core Evaluation Function

Node Type Handlers

Type	Logic
AND	All children must return `isValid: true`
OR	At least one child must return `isValid: true`
COURSE	Check `transcript.completedCourses.includ`
YEAR	Compare `transcript.currentYear >= requiredYear`
PERM	Check `transcript.hasSpecialPermission`

Complexity Analysis

```
async function evaluatePrerequisiteNode(
  node: PrerequisiteNodeData,
  transcript: StudentTranscript,
  allNodes: PrerequisiteNodeData[]
): Promise<EvaluationResult> {

  switch (node.nodeType) {
    case 'AND': {
      const children = allNodes.filter(
        n => n.parentId === node.id
      )
      const results = await Promise.all(
        children.map(c =>
          evaluatePrerequisiteNode(c, transcript, allNodes)
        )
      )
      const failed = results.filter(r => !r.isValid)
      return {
        isValid: failed.length === 0,
        missingItems: failed.flatMap(r => r.missingItems)
      }
    }

    case 'OR': {
      // Similar but returns true if ANY child passes
    }

    case 'COURSE': {
      // Check if course in transcript
    }
  }
}
```

- **Time:** O(n) where n = number of nodes
- **Space:** O(h) where h = tree height (recursion stack)
- Typical prerequisite tree: 5-15 nodes

Prerequisite Validation: Grade Requirements

Some courses require a minimum grade, not just completion

Grade Scale Implementation

```
function meetsGradeRequirement(
  actualGrade: string | undefined,
  requiredGrade: string
): boolean {
  if (!actualGrade) return false

  const gradeValues: Record<string, number> = {
    'A+': 12, 'A': 11, 'A-': 10,
    'B+': 9,  'B': 8,  'B-': 7,
    'C+': 6,  'C': 5,  'C-': 4,
    'D+': 3,  'D': 2,  'D-': 1,
    'F': 0
  }

  const actualValue = gradeValues[actualGrade]
  const requiredValue = gradeValues[requiredGrade]

  // Unknown grades: be permissive
  if (actualValue === undefined ||
      requiredValue === undefined) {
    return true
  }

  return actualValue >= requiredValue
}
```

Usage in COURSE Node Evaluation

```
case 'COURSE': {
  const hasCourse = transcript.completedCourses
    .includes(node.requiredCourseId)

  if (!hasCourse) {
    return {
      isValid: false,
      reason: `Missing ${node.requiredCourseId}`,
      missingItems: [node.requiredCourseId]
    }
  }

  // Check grade if required
  if (node.requiredGrade) {
    const actual = transcript.courseGrades?.[
      node.requiredCourseId
    ]
    if (!meetsGradeRequirement(actual, node.requiredGrade)) {
      return {
        isValid: false,
        gradeIssues: [`${node.requiredCourseId}: need ${node.requiredGrade},
have ${actual}`]
      }
    }
  }

  return { isValid: true }
}
```

Prerequisite Validation: User Feedback

Providing actionable information, not just pass/fail

ValidationResult Interface

```
interface PrerequisiteValidationResult {
  isValid: boolean
  reason?: string
  missingCourses?: string[]    // ["SYSC3303", "SYSC3310"]
  yearRequirement?: string     // "FOURTH_YEAR"
  needsPermission?: boolean
  gradeIssues?: string[]       // ["COMP1006: need C-, have D+"]
  expressionEvaluated: string  // Original expression
}
```

We return **all** missing items, not just the first one found. This helps students plan which courses to take.

Example Output

Course: SYSC 4001
Status: FAIL
Required: `SYSC3303 && (SYSC3310 || SYSC3320)`
Missing courses: SYSC3303
Details: You need SYSC3303 AND at least one of SYSC3310/SYSC3320

Course: SYSC 4907
Status: PASS
Required: `FOURTH_YEAR`
Details: You meet all prerequisites

Section 3: Schedule Generation

Finding conflict-free timetables using backtracking

The Combinatorial Problem

A typical term schedule involves:

- 5 courses
- 3-4 sections per course
- Each section may have LEC + LAB + TUT

Worst case: 4 sections × 5 courses = 4^5 = **1,024 combinations**
With LEC/LAB/TUT: easily **10,000+ combinations**

Manual checking is tedious and error-prone. We need an algorithm.

Requirements

1. **Find ALL valid schedules** — not just one
2. **No time conflicts** — classes can't overlap
3. **Include all components** — LEC + LAB + TUT
4. **Fast enough** — user shouldn't wait > 1-2 seconds
5. **Bounded output** — don't return 10,000 schedules

Algorithm Choice: Backtracking

- Systematic search of solution space
- **Prune early** when conflict detected
- Guaranteed to find all solutions
- Much faster than brute force in practice

Schedule Generation: Data Structures

TypeScript interfaces for sections, schedules, and time slots

Section Interface

```typescript
interface Section {
  id: string
  courseId: string      // "SYSC3303"
  crn: string | null    // "12345"
  sectionCode: string   // "A"
  sectionType: string   // "LEC" | "LAB" | "TUT"
  instructor: string | null
  days: string | null   // "MWF" or "TR"
  startTime: string | null  // "0835"
  endTime: string | null    // "0925"
  location: string | null
  capacity: number | null
  enrolled: number | null
}
```

Schedule & TimeSlot

```typescript
interface TimeSlot {
  day: string       // "Monday"
  startTime: string // "0835"
  endTime: string   // "0925"
}

interface ScheduleSelection {
  courseId: string
  section: Section
}

interface GeneratedSchedule {
  id: number
  selections: ScheduleSelection[]
  hasConflicts: boolean
  prerequisiteInfo?: {
    [courseId: string]: {
      isValid: boolean
      missingCourses?: string[]
    }
  }
}
```

Schedule Generation: Time Handling

Converting database formats to comparable time slots

Day Abbreviation Expansion

```
function parseDays(daysStr: string | null): string[] {
  if (!daysStr) return []

  const dayMap: Record<string, string> = {
    'M': 'Monday',
    'T': 'Tuesday',
    'W': 'Wednesday',
    'R': 'Thursday',  // R for thuRsday
    'F': 'Friday'
  }

  return daysStr.split('')
    .map(d => dayMap[d] || d)
    .filter(d => d)
}

// "MWF" → ["Monday", "Wednesday", "Friday"]
// "TR"  → ["Tuesday", "Thursday"]
```

Section → TimeSlots

```
function getTimeSlots(section: Section): TimeSlot[] {
  if (!section.days || !section.startTime ||
      !section.endTime) {
    return []  // Online/async sections
  }

  const days = parseDays(section.days)
  return days.map(day => ({
    day,
    startTime: section.startTime!,
    endTime: section.endTime!
  }))
}

// Section with days="MWF", start="0835", end="0925"
// Returns:
// [
//   { day: "Monday",    startTime: "0835", endTime: "0925" },
//   { day: "Wednesday", startTime: "0835", endTime: "0925" },
//   { day: "Friday",    startTime: "0835", endTime: "0925" }
// ]
```

Schedule Generation: Conflict Detection

The O(1) interval overlap check at the heart of the algorithm

Time Conflict Logic

```typescript
function hasTimeConflict(
  slot1: TimeSlot,
  slot2: TimeSlot
): boolean {
  // Different days = no conflict
  if (slot1.day !== slot2.day) return false

  // Parse "0835" → 515 minutes since midnight
  const parseTime = (time: string): number => {
    const hours = parseInt(time.substring(0, 2))
    const minutes = parseInt(time.substring(2, 4))
    return hours * 60 + minutes
  }

  const start1 = parseTime(slot1.startTime)
  const end1 = parseTime(slot1.endTime)
  const start2 = parseTime(slot2.startTime)
  const end2 = parseTime(slot2.endTime)

  // Interval overlap formula
  return start1 < end2 && start2 < end1
}
```

Why This Formula Works

```
Overlap if: start1 < end2 AND start2 < end1

Case 1: [===1===]
              [===2===]  ✓ Overlap

Case 2: [===1===]
                  [===2===]  ✗ No overlap

Case 3:       [===1===]
        [===2===]  ✓ Overlap
```

Time Complexity: O(1) per comparison
Called: Many times during backtracking search

Schedule Generation: Section Grouping

Handling courses with multiple component types (LEC + LAB + TUT)

Group by Section Type

```
function groupSectionsByType(
  sections: Section[]
): Record<string, Section[]> {
  const grouped: Record<string, Section[]> = {}

  for (const section of sections) {
    if (!grouped[section.sectionType]) {
      grouped[section.sectionType] = []
    }
    grouped[section.sectionType].push(section)
  }

  return grouped
}

// Input: [LEC-A, LEC-B, LAB-L1, LAB-L2, TUT-T1]
// Output: {
//   "LEC": [LEC-A, LEC-B],
//   "LAB": [LAB-L1, LAB-L2],
//   "TUT": [TUT-T1]
// }
```

Generate All Combinations

```
function generateSectionCombinations(
  courseId: string,
  sectionsByType: Record<string, Section[]>
): Section[][] {
  const types = Object.keys(sectionsByType)
  if (types.length === 1) {
    return sectionsByType[types[0]].map(s => [s])
  }

  const combinations: Section[][] = []

  function build(typeIdx: number, current: Section[]) {
    if (typeIdx === types.length) {
      combinations.push([...current])
      return
    }
    for (const section of sectionsByType[types[typeIdx]]) {
      current.push(section)
      build(typeIdx + 1, current)
      current.pop()
    }
  }

  build(0, [])
  return combinations
}
```

2 LECs × 2 LABs × 1 TUT = 4 combinations per course

Schedule Generation: Backtracking Algorithm

The recursive search with early pruning

```
function generateSchedulesRecursive(
  courseSections: { courseId: string; sections: Section[] }[],
  currentIndex: number,
  currentSelections: ScheduleSelection[],
  maxSchedules: number,
  result: GeneratedSchedule[]
): void {
  // Base case: all courses scheduled → found valid schedule
  if (currentIndex === courseSections.length) {
    result.push({ id: result.length + 1, selections: [...currentSelections], hasConflicts: false })
    return
  }

  // Early termination: enough schedules found
  if (result.length >= maxSchedules) return

  const { courseId, sections } = courseSections[currentIndex]
  const sectionsByType = groupSectionsByType(sections)
  const combinations = generateSectionCombinations(courseId, sectionsByType)

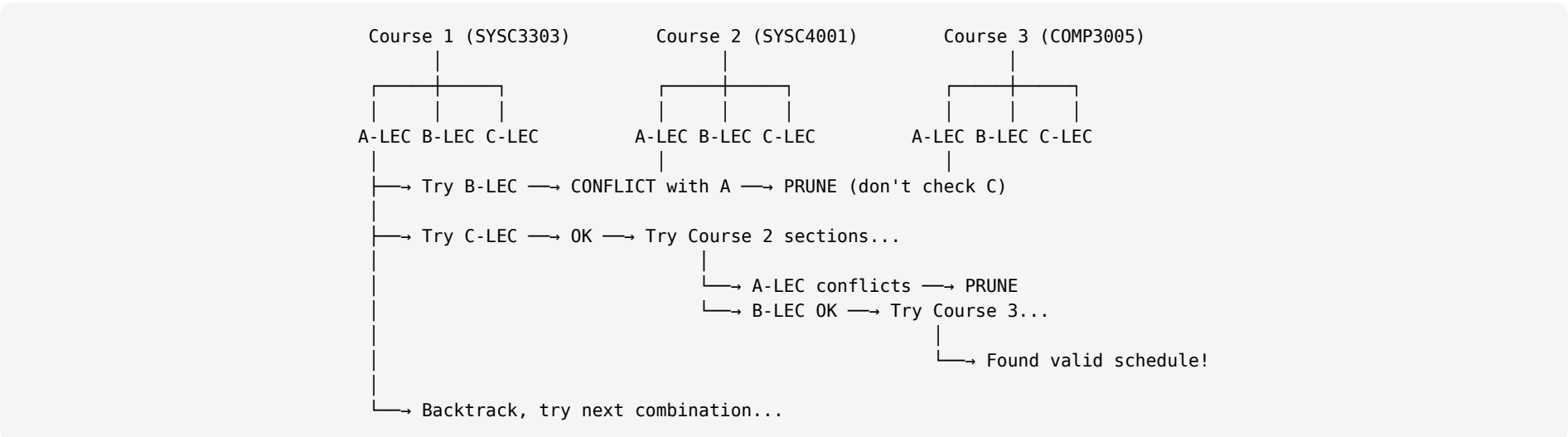
  for (const sectionCombination of combinations) {
    // Skip if LEC conflicts with its own LAB
    if (hasInternalConflicts(sectionCombination)) continue

    // Check against already-selected sections
    let hasConflict = false
    for (const section of sectionCombination) {
      if (hasConflictWithSelections(section, currentSelections)) {
        hasConflict = true
        break  // PRUNE: don't explore this branch further
      }
    }

    if (!hasConflict) {
      // Add sections and recurse
      for (const section of sectionCombination) {
        currentSelections.push({ courseId, section })
      }
      generateSchedulesRecursive(courseSections, currentIndex + 1, currentSelections, maxSchedules, result)
      // Backtrack
      for (let i = 0; i < sectionCombination.length; i++) currentSelections.pop()
    }
  }
}
```

Schedule Generation: Algorithm Visualization

How backtracking explores and prunes the search tree



Why Pruning Helps

- Conflict at Course 1 ⟶ skip all Course 2/3 combinations
- Exponential reduction in practice
- Most branches get pruned early

Complexity

- **Worst case:** O(b^d) where b=sections, d=courses
- **Average case:** Much better due to pruning
- **Real performance:** < 1 second for 5 courses

Schedule Generation: API Integration

REST endpoint with prerequisite validation

POST /schedules/generate

Response Format


```
{
  "schedules": [
    {
      "id": 1,
      "selections": [
        {
          "courseId": "SYSC3303",
          "section": {
            "crn": "12345",
            "sectionCode": "A",
            "days": "MWF",
            "startTime": "0835",
            "endTime": "0925"
          }
        }
      ],
      "hasConflicts": false,
      "prerequisiteInfo": {
        "SYSC3303": { "isValid": true }
      }
    }
  ],
  "count": 15,
  "maxSchedules": 50
}
```


Frontend displays these as timetable grids with CRNs for easy registration.

```
router.post('/generate', async (req, res) => {
  const {
    courseIds,           // ["SYSC3303", "SYSC4001"]
    term,                // "Winter 2026"
    maxSchedules = 50,   // Cap results
    completedCourses = [] // For prereq check
  } = req.body

  // Validation
  if (courseIds.length > 8) {
    return res.status(400).json({
      error: 'Maximum 8 courses allowed'
    })
  }

  // Fetch sections from DB
  const courseSections = await Promise.all(
    courseIds.map(async (id) => ({
      courseId: id,
      sections: await prisma.section.findMany({
        where: { courseId: id, term }
      })
    }))
  )

  // Run backtracking
  const schedules: GeneratedSchedule[] = []
  generateSchedulesRecursive(
    courseSections, 0, [], maxSchedules, schedules
  )

  // Add prerequisite info to each schedule
  for (const schedule of schedules) {
    schedule.prerequisiteInfo = await checkPrereqs(...)
  }

  res.json({ schedules, count: schedules.length })
})
```

Section 4: Data Acquisition

Populating the database from scattered university sources

The Data Challenge

Carleton has **no public API**. Data is scattered across:

Data	Source
Course titles	Calendar HTML pages
Prerequisites	Calendar text descriptions
Section times	Timetable system
Elective lists	Department PDFs

Each source has different formats, update schedules, and access methods.

Our Approach

Separation of concerns:
- CLI scripts handle data ingestion
- Web app only reads from database
- Scripts are idempotent (safe to re-run)

Scripts Overview

1. `seed.ts` — Course data + prerequisites
2. `import-timetable.ts` — Section schedules
3. `fetch-all-course-titles.ts` — Course titles
4. `merge-course-data.ts` — Combine sources

Data Acquisition: Database Schema

Prisma models for courses, sections, and relationships

Course Model

```
model Course {
  id          String   @id  // "SYSC3303"
  title       String?       // "Real-Time..."
  description String?
  credits     Float?

  // Relations
  sections    Section[]
  prerequisiteNodes PrerequisiteNode[]
  prerequisiteExpression Prerequisite?
  courseCompletions CourseCompletion[]

  @@map("courses")
}
```

Section Model

```
model Section {
  id          String   @id @default(cuid())
  crn         String?
  courseId    String
  sectionCode String   // "A", "B", "L1"
  sectionType String   // "LEC", "LAB", "TUT"

  // Schedule
  days        String?  // "MWF"
  startTime   String?  // "0835"
  endTime     String?  // "0925"
  term        String?  // "Winter 2026"

  // Capacity
  capacity    Int?
  enrolled    Int?
  status      String?  // "Open", "Closed"

  course      Course   @relation(fields: [courseId])

  @@index([courseId])
  @@index([term])
  @@map("sections")
}
```

Data Acquisition: Course Seeding

Two-pass approach for foreign key constraints

Pass 1: Create All Courses

```
// Read prerequisites JSON
const data = JSON.parse(
  await fs.readFile('course_prerequisites_simple.json')
)

// Collect all course codes (including prereq refs)
const allCourses = new Set<string>()
for (const [courseId, entry] of Object.entries(data)) {
  allCourses.add(courseId)

  // Extract courses from prerequisite expression
  const prereqCourses = entry.required?.match(
    /\b[A-Z]{3,4}\d{4}\b/g
  ) || []
  prereqCourses.forEach(c => allCourses.add(c))
}

// Create all course records first
for (const courseId of allCourses) {
  await prisma.course.upsert({
    where: { id: courseId },
    update: {},
    create: {
      id: courseId,
      title: data[courseId]?.title || courseId
    }
  })
}
```

Pass 2: Create Prerequisites

```
for (const [courseId, entry] of Object.entries(data)) {
  if (entry.required) {
    // Store human-readable expression
    await prisma.prerequisite.upsert({
      where: { courseId },
      update: { expression: entry.required },
      create: {
        courseId,
        expression: entry.required
      }
    })

    // Clear old nodes
    await prisma.prerequisiteNode.deleteMany({
      where: { courseId }
    })

    // Parse and create tree
    const parsed = parsePrerequisiteExpression(
      entry.required
    )
    await createPrerequisiteTree(courseId, parsed)
  }
}
```

Why two passes? PrerequisiteNode has FK to Course. All courses must exist before we can reference them.

Data Acquisition: Timetable Import

Parsing tab-separated timetable data

Input Format (TSV)

```
term  sectionId  dept  num  code  type  days  start  end  ...
25WN  12345      SYSC  3303 A     LEC   MWF   0835   0925 ...
25WN  12346      SYSC  3303 L1    LAB   T     1435   1725 ...
```

Import Process

Parsing Logic

```
function parseTimetableLine(line: string) {
  const parts = line.split('\t')
  if (parts.length < 14) return null

  const [term, sectionId, dept, num,
         code, type, days, start, end,
         startDate, endDate, cap, enrolled, status] = parts

  return {
    term,
    courseId: `${dept}${num}`,  // "SYSC3303"
    sectionCode: code,
    sectionType: type,
    days: days || '',
    startTime: start || '',
    endTime: end || '',
    capacity: parseInt(cap) || 0,
    enrolled: parseInt(enrolled) || 0,
    status
  }
}
```

```
async function main() {
  const content = await fs.readFile(
    'scheduling-timings.txt', 'utf-8'
  )
  const lines = content.split('\n').filter(l => l.trim())

  for (const line of lines) {
    const record = parseTimetableLine(line)
    if (!record) continue

    // Ensure course exists
    await prisma.course.upsert({
      where: { id: record.courseId },
      update: {},
      create: { id: record.courseId, title: record.courseId }
    })

    // Create section
    await prisma.section.create({
      data: {
        courseId: record.courseId,
        sectionCode: record.sectionCode,
        sectionType: record.sectionType,
        days: record.days,
        startTime: record.startTime,
        endTime: record.endTime,
        term: record.term,
        capacity: record.capacity,
        enrolled: record.enrolled
      }
    })
  }
}
```

Data Acquisition: Elective Categories

Mapping elective requirements to valid course lists

Category Definitions

```
export const SYSC_ELEC_ELECTIVES = [
  // SYSC 3000-level
  'SYSC3006', 'SYSC3010', 'SYSC3200', ...
  // SYSC 4000-level
  'SYSC4005', 'SYSC4111', 'SYSC4130', ...
  // ELEC options
  'ELEC3105', 'ELEC4505', ...
]

export const CS_ELECTIVES = [
  'COMP2401', 'COMP3000', 'COMP3004',
  'COMP4107', 'COMP4601', ...
]

export const COMPLEMENTARY_STUDIES = [
  'PHIL1200', 'SOCI1001', 'PSYC1001',
  'ECON1000', 'BUSI1001', 'CCDP2100', ...
]

export const ELECTIVE_CATEGORIES = {
  'SYSC_ELEC_ELECTIVE': SYSC_ELEC_ELECTIVES,
  'CS_ELECTIVE': CS_ELECTIVES,
  'COMPLEMENTARY_STUDIES': COMPLEMENTARY_STUDIES,
  'SCIENCE_ELECTIVE': SCIENCE_ELECTIVES
}
```

Expansion Function

```
export function expandElectiveCategories(
  courseOptions: string[]
): string[] {
  const expanded: string[] = []

  for (const option of courseOptions) {
    if (ELECTIVE_CATEGORIES[option]) {
      // Category token → expand to course list
      expanded.push(...ELECTIVE_CATEGORIES[option])
    } else {
      // Regular course code → keep as-is
      expanded.push(option)
    }
  }

  return [...new Set(expanded)]  // Deduplicate
}

// Example:
// Input:  ["CS_ELECTIVE", "SYSC4907"]
// Output: ["COMP2401", "COMP3000", ..., "SYSC4907"]
```

Used by audit parser to show which courses can fulfill elective requirements.

Data Acquisition: Idempotent Design

Scripts that are safe to run multiple times

What is Idempotent? Running the script once = running it N times

Same final state, no duplicates, no errors

Techniques Used

Operation	How
Create/Update	`prisma.upsert()`
Clear old data	`deleteMany()` before insert
Unique keys	Primary keys prevent duplicates
Check existence	`findUnique()` before create

Upsert Pattern

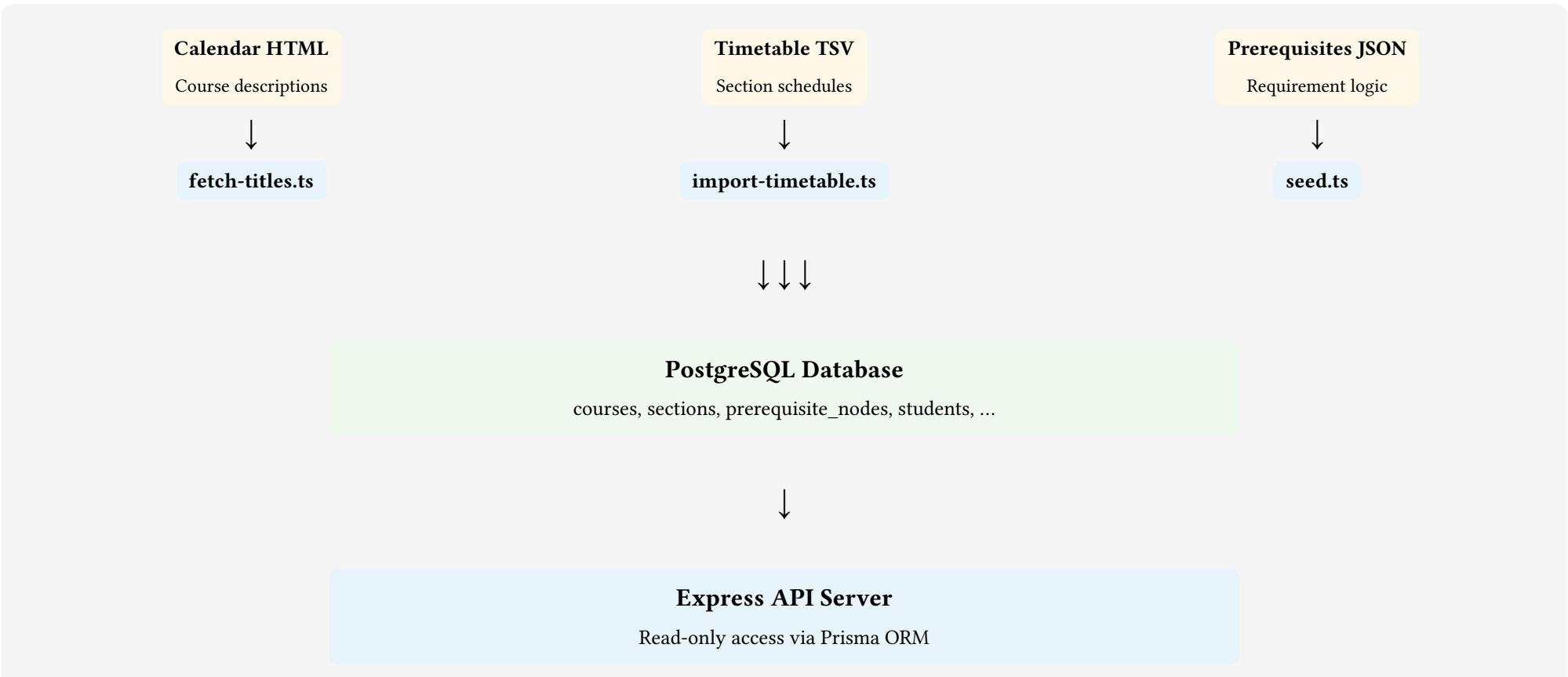
```
// Safe to run repeatedly
await prisma.course.upsert({
  where: { id: courseId },
  update: {
    title: newTitle,
    description: newDescription,
    updatedAt: new Date()
  },
  create: {
    id: courseId,
    title: newTitle,
    description: newDescription
  }
})
```

Why This Matters

- Data updates each term
- Want to refresh without manual cleanup
- Mistakes don't corrupt database
- Easy to test and debug

Data Acquisition: Pipeline Architecture

How all the pieces fit together



Data Acquisition: Future Work

Planned improvements for data freshness and coverage

Current Limitations

1. **Manual elective lists** — currently hardcoded arrays, should come from official sources
2. **Single term at a time** — no Fall+Winter planning
3. **No enrollment status updates** — sections may fill up
4. **Manual refresh** — no automated data updates

Planned Improvements

1. **Automated elective ingestion** — parse department PDFs or scrape official lists
2. **Multi-term support** — Fall/Winter/Summer selection in UI
3. **Periodic refresh job** — scheduled script to update section data
4. **Admin interface** — web UI for manual data corrections

Key insight: Separating data acquisition from the app means we can improve the pipeline without changing the frontend or API.

Audit Parsing: Reliability & Trade-offs

Fast parsing with fallbacks for edge cases

Approaches Considered

Option	Trade-off
DOM parser (cheerio/ jsdom)	More robust, heavier dependency
Regex + fallbacks	Fast + simple, brittle if HTML changes

What We Implemented

- targeted patterns for stable fields
- fallback handling for missing grades/rows
- O(n) scan over the HTML

Example Patterns (Simplified)

```
/(\d{9})/                          → student ID
/Bachelor of Engineering - ([^<]+)/ → program
/<tr.*class="takenCourse".*>/g     → course rows
```

Known limitation: if Carleton changes page structure, we'll need to adjust patterns (or switch to a DOM parser).

Implementation 2/4: Prerequisite Representation

Store AND/OR logic once, evaluate many times

The Core Problem Prerequisites are logical expressions, not simple lists:

```
SYSC 4001 needs:
(SYSC 3303 AND SYSC 3310) OR permission
Fourth-year standing OR (AERO 2001 AND MATH 2004)
```

Implementation Goal Avoid re-parsing strings on every request. Store the structure in the DB so it's reusable and queryable.

Chosen Model: Tree Nodes

```
        [OR]
       /    \
   [AND]    [PERM]
   /   \
3303  3310

Node types: AND | OR | COURSE | YEAR | PERMISSION
```

Result: prerequisite checking becomes a fast tree walk

Prerequisite Evaluation: How It Runs

Recursive evaluation against the student's completions

Algorithm

```
eval(node):
  AND  → all children pass
  OR   → any child passes
  COURSE → course in transcript (and grade if required)
  YEAR → student standing >= required
  PERM → requires manual override
```

Complexity

- Time: O(n) for n nodes
- Space: O(h) recursion depth

Where It's Used

- course search / course detail: show "eligible vs missing prereqs"
- schedule building: filter out courses you can't take

Trade-off: a tree schema is more complex than a string, but it prevents repeated parsing and keeps the logic consistent everywhere.

Implementation 3/4: Schedule Generation

Generate conflict-free combinations automatically

Why This Needs an Algorithm A typical term quickly becomes combinatorial:

- 5 courses
- multiple sections per course
- LEC/LAB/TUT constraints

Goal (Implemented) Find many valid schedules (not just one) and stop early when a partial choice already conflicts.

Chosen Approach: Backtracking

```
pick course 1 section
  → try course 2 sections
    → if conflict: prune branch
    → else continue
```

Optimization: cap results (e.g., 50) to bound runtime

Schedule Conflicts: The Constraint

Detect time overlaps quickly while building schedules

Conflict Rule

```
Two meetings conflict iff:
same_day AND start1 < end2 AND start2 < end1
```

Why It Works

- O(1) overlap check
- called repeatedly during the backtracking search

What a "Section" Means

We treat a selected option as a bundle of required components:

- LEC must fit
- and LAB/TUT (if required) must fit

Trade-off: backtracking guarantees correctness, but worst-case runtime grows fast — pruning + caps keep it practical.

Implementation 4/4: Data Acquisition

Populate the DB from scattered Carleton sources

Reality Check There's no single public API:

- course calendar: many static HTML pages
- timetable: separate system
- electives: program-specific PDFs

Design Choice

Separate data ingestion from the web app (CLI/scripts populate DB; app only reads)

Pipeline (Implemented)

Data		How We Get It
Course titles + descriptions	←	parse calendar HTML pages
Prerequisite text	←	extract + convert to node tree
Section meeting times	←	import timetable export

Idempotent imports: rerun scripts without duplicating data

Data Evolution: What's Next

Keep data fresh each term and expand electives

In Place Today

- course + prerequisite ingestion into PostgreSQL
- term-aware section storage (Fall/Winter)
- clear split: scripts write, app reads

Constraints

- data changes every term
- must avoid hammering university systems

Remaining Work

- automate elective list ingestion
- improve multi-term (Fall/Winter/Summer) support
- admin tools for refresh + validation

This keeps the app accurate without manual re-entry each year.

Design Principle: User-Centered Design

Making It Simple — Before vs After

The Old Way

"Can I take SYSC 4001?"
⟶ Open audit in one tab
⟶ Open calendar in another
⟶ Find the prerequisites
⟶ Check each one against your audit
⟶ Figure out the AND/OR logic

Repeat for every course

With Our App

"Can I take SYSC 4001?"
⟶ Type "SYSC 4001"
⟶ See "You're good" or "Need SYSC 3303"

Instant feedback

Upload once, and the app remembers what you've completed

Design Principle: Constraint Satisfaction

Too many combinations to check by hand

The Problem

- Pick 5 courses
- Each course may have multiple sections
- Each section may have LEC, LAB, TUT components
- All combinations need to be checked for time conflicts

Doing this manually is tedious and error-prone.

How We Do It: Backtracking

```
generateSchedules(courses, current):
  if all courses scheduled:
    save it, return
  for section in nextCourse:
    if no conflict:
      add section, recurse, remove
    else: skip this branch
```

If something conflicts, stop early and try the next option. This saves time early.

Design Principle: Separation of Concerns

Three layers, each does one job

Frontend — React + TypeScript

What you see and interact with

↓ talks to ↓

Backend — Express + Node.js

Handles requests, runs the logic, talks to the database

↓ queries ↓

Database — PostgreSQL

Stores courses, sections, and prerequisite rules

Design Principle: Data Modeling

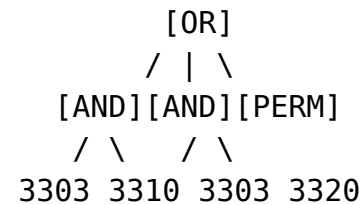
Prerequisites aren't just a list — they're logic

The Problem

Prerequisites have AND/OR conditions:

> SYSC 4001 needs:
> (SYSC 3303 AND SYSC 3310)
> OR (SYSC 3303 AND SYSC 3320)
> OR (Permission of instructor)

Our Solution: A Tree



Walk the tree, check each node. If any OR branch passes, you're good.

Design Principle: Performance Optimization

Don’t hammer the database

Without Optimization

User types “SYSC4001”
⟶ 8 keystrokes = 8 API calls
⟶ 8 database queries
Multiply by many concurrent users…

What We Do Instead

Debounce — wait 300ms after typing stops
Cache — remember results for 5 min
Min length — don’t search for “a”
Cancel — abort stale requests

Scenario	Before	After
Typing “SYSC4907”	8 requests	1 request
Same search twice	2 requests	0 (cached)

Work in Progress: Electives

The Annoying Part

Right now if you want to know what Science Electives you can take as a Software Eng student, you have to go to a separate website and dig through PDFs.

Same deal for Complementary Studies, tech electives, etc.

Where We're At

- We detect elective types from your audit
- Currently using placeholder data

What We're Adding

1. Add elective tables to the DB
Which electives count for which programs

2. Get the real data
From Carleton's official lists

3. Suggest electives automatically
Based on your program and year

4. Include them in schedule generation
Pick a Science Elective → see all options

Tech Stack

Layer	Tech	Why
Frontend	React + TypeScript	Components, type safety
Styling	TailwindCSS	Fast to write, looks consistent
Backend	Express + Node.js	Simple, fast, JS everywhere
Database	PostgreSQL	Solid, handles complex queries
ORM	Prisma	Type-safe DB queries, easy migrations

How It Works

From upload to registration

1. Upload

Drop in your audit HTML

→

2. Parse

We extract your courses

→

3. Search

Find courses, check prereqs

4. Generate

Click the button

→

5. Pick One

Browse the timetables

→

6. Register

Copy CRNs, paste in Central

Problems We Ran Into

Problem	How We Solved It
Prerequisites have complex AND/OR logic	Store them as trees, walk recursively
Many schedule combinations to check	Backtracking with early exit on conflicts
Search hitting DB too much	Debounce, cache, cancel old requests
Audits aren't all formatted the same	Pattern matching with fallbacks
Page refresh loses your state	URL routing + sessionStorage
Checking time conflicts	Interval overlap check, filter by day first

What's Left (2 Months)

Electives (Main Focus)

- Get official elective lists into the DB
- Link them to programs
- Auto-suggest based on your audit
- Include in schedule generation

UI Polish

- Better way to compare schedules
- Improve visual feedback

Term Selection

- Add ability to pick Fall vs Winter
- Fetch section data for both terms

Testing & Edge Cases

- End-to-end tests
- Handle weird audit formats
- Make sure it doesn't break

Summary

What It Does

- ✓ Upload your audit once, we remember it
- ✓ Search any course, see if you qualify
- ✓ Generate conflict-free schedules
- ✓ Copy CRNs in one click

How We Built It

- Focused on making it simple to use
- Backtracking to handle the combinatorics
- Clean separation: frontend / backend / DB
- Trees for prerequisite logic
- Debouncing + caching so search is fast

Thanks!

Questions?

Project: SYSC 4907 Capstone
Supervisor: Professor Jerome Talim
Stack: React, Express, PostgreSQL, Prisma, TypeScript